Why Reinforcement Learning Guesses Jump Around — and How to Calm Them

Student Name: Muthuraj Jayakumar **Course:** CSCN8020 — Reinforcement Learning · **Date:** 19 June 2026

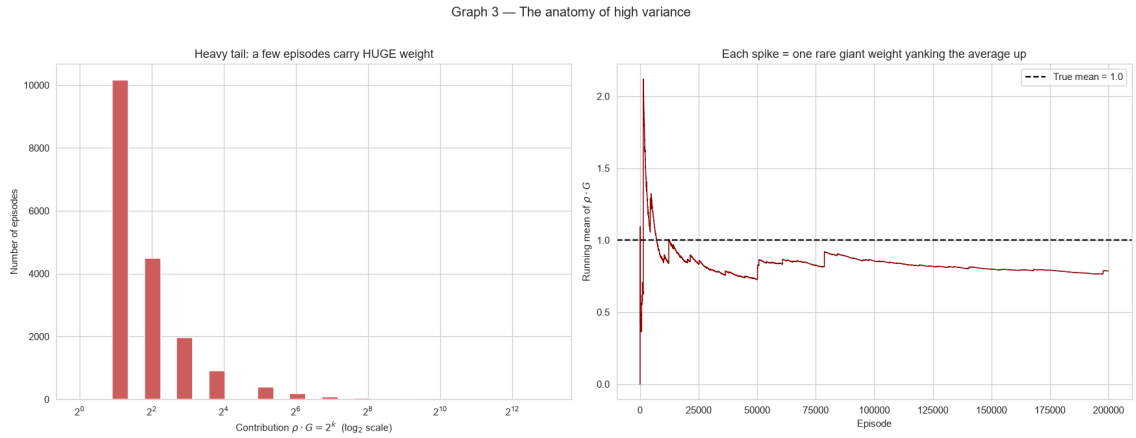
What this report is about

In class we did three small experiments. They all look at the same problem: when a computer learns just by trying things and watching what happens, its guesses can **jump around a lot**. If I train it again from scratch, I can get a very different answer. That jumpiness is called **variance**. High variance is bad because I can't trust any single run.

The nice thing about these exercises is that each one has a **known correct answer** (the bullseye, or "true value"), so I can actually measure how jumpy each method is and whether it lands on the truth. For each exercise below I show the one graph that puts the jumpiness and the true answer side by side.

Exercise 1 — Learning from someone else's random moves

Here the agent learns about a good plan by watching a completely random explorer and then re-weighting what it saw ("how likely would *my* plan have made that same move?"). Those weights get multiplied step after step, so every once in a while one lucky run earns a giant weight. The true value it should land on is **1.0**.

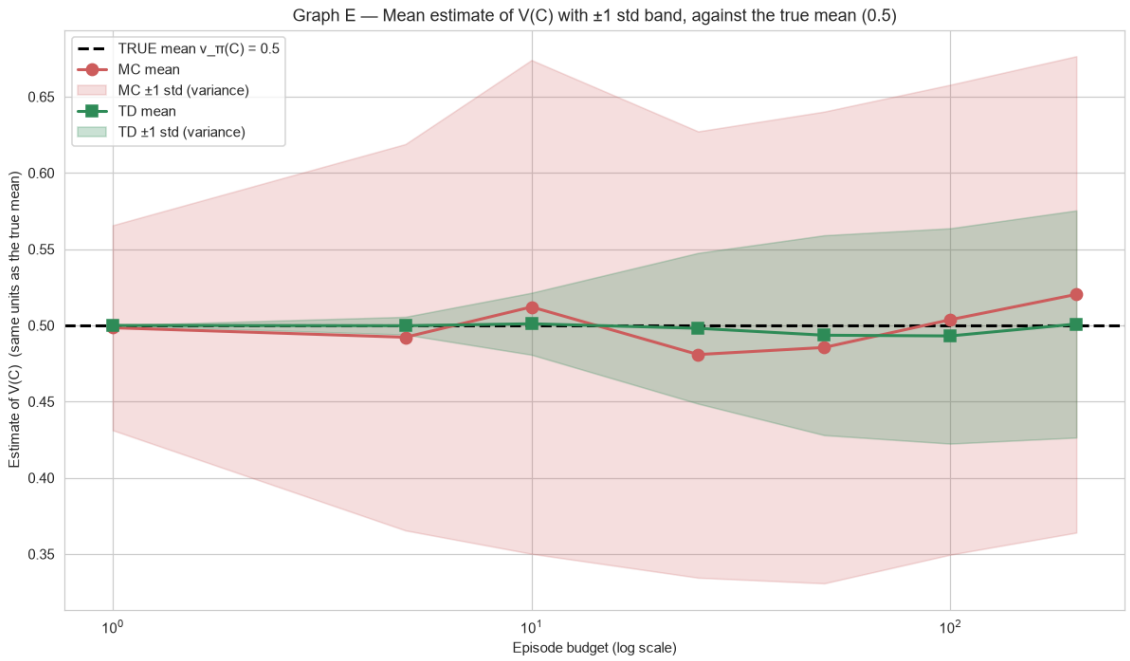


Left: most runs carry a small weight (2, 4, 8…), but a few carry enormous ones — a "heavy tail." Right: the running average (red) keeps getting yanked upward every time one of those rare giant weights shows up, and never settles onto the true value of 1.0 (dashed line).

What I learned: the jumpiness isn't just random bad luck — it has a clear cause. A handful of runs with monster weights dominate everything, so the average jolts up whenever one appears and then drifts, never settling on the truth. Adding more runs doesn't help, because there's always a bigger monster waiting. The fix is a self-correcting ("weighted") version that cancels these monsters out and lands calmly on 1.0.

Exercise 2 — Updating the guess step by step

Same theme, but now I compare two ways to score one fixed plan. The **averaging way (Monte Carlo)** waits for the whole episode to finish and then uses the final result. The **step-by-step way (TD)** instead nudges its guess a little at every step, using its own next guess, without waiting for the end. The true value of the middle state is **0.5**.



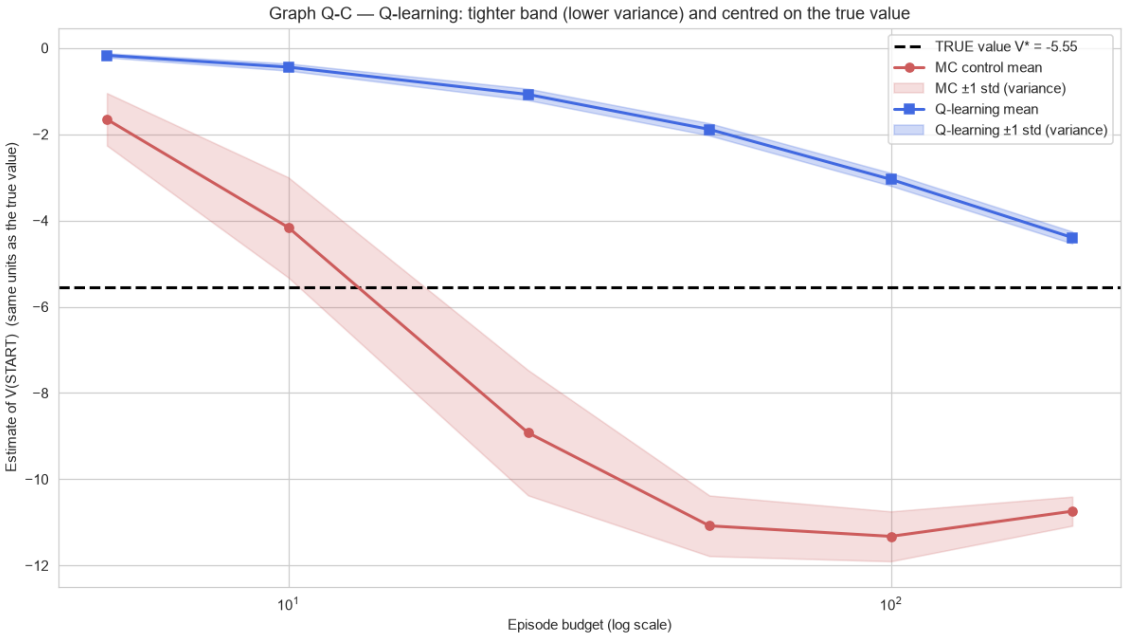
The line is each method's average guess, the shaded band shows how much it jumps (the variance), and the dashed line is the true value 0.5. The step-by-step method's band is much thinner.

What I learned: waiting for the whole episode soaks up all the luck and bad luck of the entire trip, which makes the averaging way jumpy. The step-by-step way only takes in one step of randomness at a time, so it is much steadier — and still lands on 0.5.

Exercise 3 — Finding the *best* plan (Q-learning)

Now the goal isn't just to score a plan but to **find the best one**. Q-learning is the step-by-step method for that. I tested it on a small slippery grid where the agent must reach a goal while avoiding holes. Every step costs points and a hole costs a lot, so one unlucky slip changes the score a lot — that is what creates the jumpiness. The true best score starting out is about –**5.5**, and I compared Q-learning against the averaging method.

I also did the required speed task. The slow part of the update is "look at all the possible next moves and take the best one." Writing that as a single NumPy operation instead of a plain loop gives the exact same answer but up to about **17× faster** once there are more than 100 possible actions — so it scales to big problems.



Each method's average guess (line) with its jumpiness band, against the true value (dashed). Q-learning's band is thinner and climbs onto the true value; the averaging method is wider and even settles on the wrong, too-low score.

What I learned: Q-learning is both steadier (lower variance) and more accurate here. Because it always learns from the *best* next move, it isn't fooled by its own random exploring. The averaging method keeps falling into holes while it explores, so it ends up far too pessimistic.

The big picture

It's the same story three times:

The jumpy method	The calmer fix	What the fix gives up
Plain re-weighting (Ex 1)	Self-correcting weighting	A tiny error that shrinks with data
Whole-episode averaging (Ex 2)	Step-by-step TD	A tiny error early on
Whole-episode averaging (Ex 3)	Step-by-step Q-learning	A tiny error early on

In every case, leaning on your own running guess (or a smart correction) trades a tiny bit of accuracy for a big drop in jumpiness. When you only get a limited number of runs, the calmer method wins. This simple trade is exactly what makes modern reinforcement learning — like the AI that learns to play games — actually work.

Conclusion

Across all three exercises, the methods that update step by step (TD and Q-learning), or that correct their weights, are far less jumpy than plain whole-episode averaging, and they still hit the right answer. For Q-learning I also showed the update can be made fast enough for large problems. The graphs make the point at a glance: thinner bands sitting right on the bullseye.